Master Informatique

Structures de données avancées

Projet M1

Rapport de projet

Route planning : répondre vite à des requêtes de plus court chemin

---

Réalisé par

Zeynab SALEHI
Amina BENAINI
Boboc TEODORA

Encadré par : M. Olivier BODINI

Année universitaire : 2025/2026

Contents

1 Introduction **2**

2 Données et modélisation **2**
2.1 Source des données 2
2.2 Modélisation du réseau 2
2.3 Calcul des distances (Haversine) 3
2.4 Statistiques du jeu de données 3
2.5 Génération des requêtes 3

3 Structure du graphe **3**
3.1 Représentation CSR (Compressed Sparse Row) 3
3.2 Optimisation des performances 3
3.3 Problème des identifiants 4

4 Algorithmes **4**
4.1 Dijkstra 4
4.2 A* 4
4.3 ALT 4
4.4 Contraction Hierarchies (CH) 5
4.4.1 Principe 5
4.4.2 Ordre 5
4.4.3 Ajout de raccourcis 5
4.4.4 Witness search 5
4.4.5 Fonctionnement réel de notre CH 5
4.4.6 Requête 5

5 Protocole expérimental **5**
5.1 Distribution 6

6 Résultats **6**
6.1 Temps 6
6.2 Débit 6
6.3 Prétraitement 6
6.4 Mémoire 6
6.5 Correction 6

7 Figures **7**

8 Analyse **8**

9 Tableau de synthèse **9**

10 Conclusion **9**

1 Introduction

Dans le cadre des systèmes de transport modernes et des services de géolocalisation (GPS), le calcul d'itinéraires en temps réel représente un défi algorithmique majeur. Avec l'augmentation constante de la taille des réseaux routiers numériques, comme ceux fournis par OpenStreetMap, les méthodes classiques de recherche de chemin, telles que l'algorithme de Dijkstra, atteignent rapidement leurs limites de performance lors du traitement de volumes massifs de requêtes.
L'objectif principal de ce projet est de concevoir et d'implémenter un moteur de planification d'itinéraire de haute performance. Au-delà de la simple exécution d'algorithmes, cette étude se concentre sur l'analyse comparative de plusieurs stratégies d'accélération :

- **Dijkstra (Baseline)** : établir une référence de correction et de performance
- **A* (Heuristique)** : réduire l'espace de recherche via des estimations géographiques
- **ALT (Landmarks)** : exploiter des points de repère pré-calculés
- **Contraction Hierarchies (CH)** : viser une accélération importante grâce à un prétraitement du graphe

L'originalité de notre approche réside également dans l'optimisation de la couche logicielle. Un accent particulier est mis sur la gestion de la mémoire via la structure **CSR (Compressed Sparse Row)**, afin de maximiser la localité cache et de minimiser l'empreinte mémoire sur des graphes de grande taille.
Enfin, cette étude vise à quantifier les compromis (*trade-offs*) entre le temps de prétraitement, la consommation mémoire et l'efficacité des requêtes.

2 Données et modélisation

2.1 Source des données

Les données utilisées dans ce projet proviennent de **OpenStreetMap (OSM)**, une base de données géographique libre et collaborative.
Afin de construire un graphe pertinent pour le calcul d'itinéraires routiers, un filtrage a été appliqué pour ne conserver que les axes routiers principaux (*highways*). Ce choix permet d'éliminer les chemins non pertinents pour la navigation automobile (zones piétonnes, pistes cyclables, etc.).
Le fichier brut au format `.osm.pbf` a ensuite été transformé en un format structuré (`CSV`) afin de permettre une lecture efficace dans notre implémentation en langage C.

2.2 Modélisation du réseau

Le réseau routier est modélisé sous la forme d'un graphe orienté pondéré $G = (V, E)$:

- **Nœuds (V)** : représentent les intersections routières. Chaque nœud possède un identifiant unique ainsi que des coordonnées géographiques (latitude, longitude).
- **Arêtes (E)** : représentent les segments de route reliant deux intersections. Le caractère orienté permet de modéliser les routes à sens unique.
- **Poids (w)** : représentent la distance physique entre deux nœuds, exprimée en mètres.

2.3 Calcul des distances (Haversine)

Le poids des arêtes est calculé à l'aide de la formule de Haversine :

$$d(A, B) = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

où ϕ représente la latitude, λ la longitude, et R le rayon de la Terre (environ 6371 km).
Contrairement à une distance euclidienne, cette formule prend en compte la courbure terrestre et permet d'obtenir une estimation réaliste des distances routières.
Cette distance est également utilisée comme heuristique dans l'algorithme A*.

2.4 Statistiques du jeu de données

Le graphe utilisé pour les expérimentations présente les caractéristiques suivantes :

- Nombre de nœuds : 100 000
- Nombre d'arêtes : 176 661
- Empreinte mémoire : 4.31 MB

Ce ratio d'environ 1.7 arêtes par nœud est représentatif d'un réseau routier réel.

2.5 Génération des requêtes

Un ensemble de 500 requêtes (s, t) a été généré aléatoirement afin d'évaluer les performances des algorithmes.
Une graine fixe (`srand(42)`) est utilisée pour garantir la reproductibilité des résultats.
Les requêtes couvrent différentes distances (courtes, moyennes et longues), ce qui permet d'évaluer le comportement des algorithmes dans des scénarios variés.

3 Structure du graphe

3.1 Représentation CSR (Compressed Sparse Row)

Afin d'optimiser les performances mémoire et temporelles, le graphe est stocké sous forme CSR.
Cette structure repose sur trois tableaux contigus :

- **offsets** : indique, pour chaque nœud, la position de début de ses voisins dans le tableau edges
- **edges** : contient les identifiants des voisins
- **weights** : contient le poids associé à chaque arête

Cette représentation permet un accès rapide et séquentiel aux voisins d'un nœud.

3.2 Optimisation des performances

L'utilisation du CSR apporte plusieurs avantages majeurs :

- **Réduction de la mémoire** : absence de pointeurs → structure compacte
- **Localité cache** : accès contigu → meilleure utilisation du cache CPU
- **Performance** : accélération des phases de relaxation dans Dijkstra, A* et CH

Ces propriétés sont particulièrement importantes pour les graphes de grande taille.

3.3 Problème des identifiants

Les identifiants issus d'OpenStreetMap ne sont pas continus et peuvent être très grands.
Problème :

- impossible d'indexer directement les tableaux
- risque d'erreurs mémoire (segmentation fault)
- gaspillage mémoire important

Solution :

- remapping des identifiants vers un intervalle $[0, n-1]$

Cette étape est essentielle pour permettre une utilisation correcte du format CSR et garantir des accès directs efficaces.

4 Algorithmes

4.1 Dijkstra

Algorithme de référence garantissant des résultats exacts.
Il explore tous les chemins possibles de manière systématique en utilisant une file de priorité. Cependant, il ne possède aucune heuristique, ce qui le rend lent sur de grands graphes.

4.2 A*

$$f(v) = g(v) + h(v)$$

A* améliore Dijkstra en ajoutant une heuristique $h(v)$ basée sur la distance géographique.
Pourquoi ce choix ?

- simple à implémenter
- admissible (ne surestime pas)
- efficace sur graphes routiers

Cela permet de réduire fortement le nombre de nœuds explorés.

4.3 ALT

$$h(v,t) = \max_{\ell \in L} |d(\ell, t) - d(\ell, v)|$$

ALT utilise des landmarks.
Choix des landmarks :

- sélection de 3 nœuds éloignés
- objectif : couvrir différentes zones du graphe

Principe :

- pré-calcul des distances depuis les landmarks
- heuristique plus informative que A*

Limite : fortement dépendant du choix des landmarks.

4.4 Contraction Hierarchies (CH)

4.4.1 Principe

Ajouter des raccourcis pour accélérer les requêtes.

4.4.2 Ordre

$$rank(v) = deg(v) \times 1000 + id(v)$$

4.4.3 Ajout de raccourcis

$$d(u, w) > d(u, v) + d(v, w)$$

4.4.4 Witness search

Recherche limitée de type Dijkstra :

- évite le nœud intermédiaire
- arrêt anticipé

4.4.5 Fonctionnement réel de notre CH

Contrairement à CH standard :

- les nœuds ne sont pas supprimés
- aucune hiérarchie stricte
- ajout massif de shortcuts

4.4.6 Requête

Recherche bidirectionnelle :

- forward depuis la source
- backward depuis la cible (graphe inversé)

$$A \rightarrow B \rightarrow C \leftarrow D$$

Limites :

- shortcuts redondants
- pas de restriction vers le haut
- erreurs possibles (mismatch)

5 Protocole expérimental

- 500 requêtes aléatoires
- graine fixe

5.1 Distribution

- Short : 3
- Medium : 27
- Long : 470

Majorité de requêtes longues → favorise A* et ALT.

Sur les 500 requêtes générées, 55 correspondent à des paires de nœuds non connectées, soit environ 11% des cas. Ces requêtes retournent une distance infinie (absence de chemin entre la source et la destination).

Afin de distinguer les effets liés à la structure du graphe de ceux liés aux performances des algorithmes, ces cas sont conservés dans les résultats bruts mais exclus de l'analyse des performances et des figures.

6 Résultats

6.1 Temps

Algo	Avg	p50	p95
Dijkstra	17.928	19.187	24.247
A*	9.461	9.151	20.763
ALT	142.834	4.392	37.534
CH	24.472	23.414	45.144

6.2 Débit

Algo	req/s
Dijkstra	55
A*	105
ALT	7
CH	40

6.3 Prétraitement

Algo	Temps
ALT	23.159 ms
CH	14844.753 ms

6.4 Mémoire

Structure	Mémoire
Graphe	4.31 MB
CH	5.53 MB

6.5 Correction

Des mismatches sont observés pour CH.

7 Figures

Les figures présentées ci-dessous sont basées sur des données filtrées, excluant les requêtes non connectées ainsi que les valeurs extrêmes.

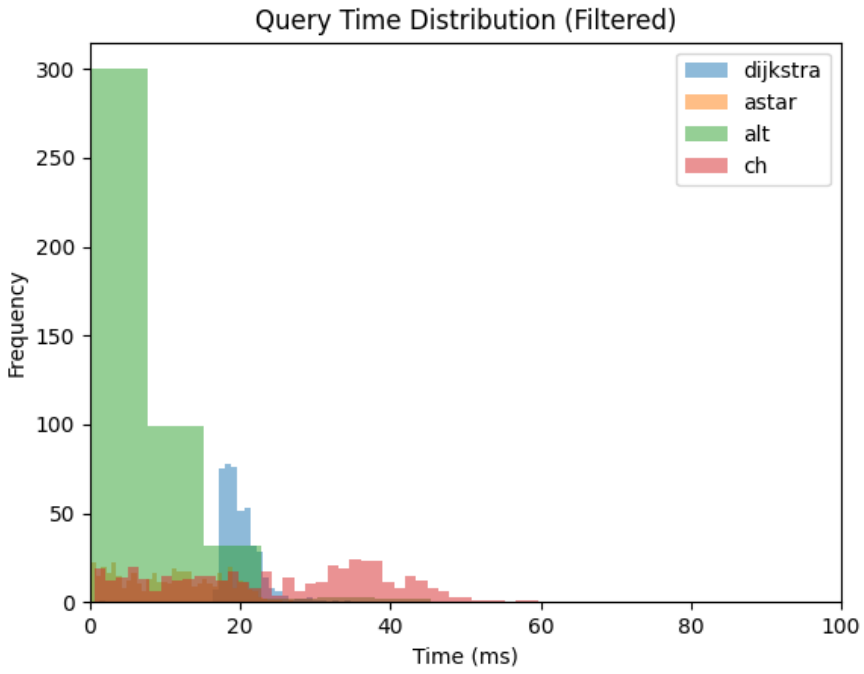

Figure 1: Distribution des temps

ALT présente une forte variabilité.

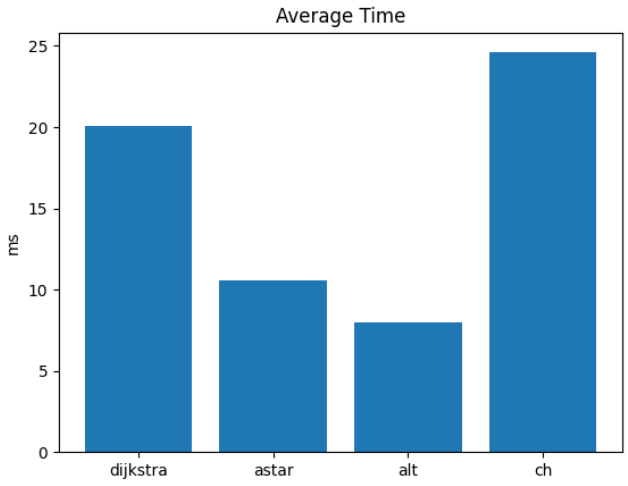

Figure 2: Temps moyen

A* est le plus efficace.

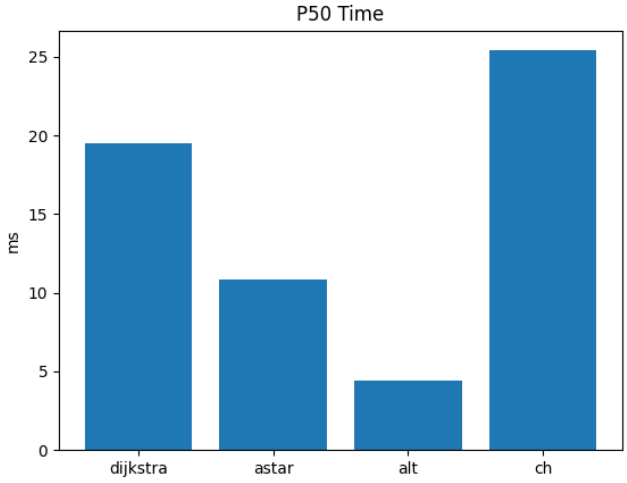

Figure 3: Temps médian

ALT est rapide dans la majorité des cas.

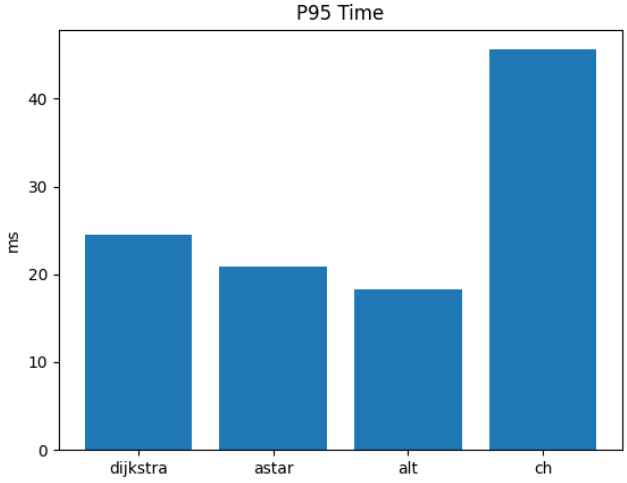

Figure 4: Cas difficiles

ALT et CH explosent sur les cas difficiles.

8 Analyse

A* offre le meilleur compromis.
ALT est instable :

- très rapide parfois
- catastrophique parfois

CH :

- prétraitement énorme
- résultats incorrects

9 Tableau de synthèse

Critère	Dijkstra	A*	ALT	CH
Exactitude	Oui	Oui	Oui	À corriger
Temps	Lent	Rapide	Variable	Moyen
Prétraitement	Aucun	Aucun	Faible	Très élevé
Mémoire	Faible	Faible	Moyenne	Moyenne

10 Conclusion

Cette étude nous a permis de quantifier l'efficacité des différentes stratégies d'accélération pour le calcul d'itinéraires sur un graphe routier réel. Nos expérimentations confirment que l'optimisation d'un moteur de recherche ne repose pas uniquement sur l'algorithme choisi, mais également sur une synergie étroite avec la structure de données sous-jacente.
Les résultats de nos benchmarks mènent aux conclusions suivantes :

- **A* constitue le meilleur compromis :** Bien qu'il ne soit pas théoriquement le plus rapide, il offre des performances stables, sans coût mémoire supplémentaire ni phase de prétraitement. Il s'agit ainsi de la solution la plus robuste dans un contexte pratique.
- **ALT dépend fortement de la sélection des landmarks :** L'efficacité de cet algorithme s'est révélée très variable. Une mauvaise sélection des points de repère peut dégrader fortement les performances, tandis qu'une sélection pertinente nécessite un prétraitement et un stockage mémoire supplémentaires.
- **CH nécessite une implémentation complète et rigoureuse :** Bien que cette approche soit théoriquement la plus performante pour les longues distances, son efficacité dépend fortement de la qualité du prétraitement (notamment le witness search) et de la gestion des raccourcis. Dans notre implémentation simplifiée, ces limitations entraînent des performances inférieures et des erreurs de calcul.

De manière générale, ce projet met en évidence l'importance des compromis entre complexité algorithmique, coût mémoire et performance réelle, en particulier dans le cadre de graphes de grande taille issus de données réelles.